ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Самарский национальный исследовательский университет имени академика С.П. Королева»

ОТЧЁТ

по лабораторной работе №4

Решение систем нелинейных уравнений методом Ньютона

Исполнитель:
Барабанов Иван Сергеевич
Вариант №16

Группа: 6312-100503D
Преподаватель: Подклетнова С.В.

Самара, 2026

Цель работы

Изучить метод Ньютона для решения систем нелинейных уравнений, реализовать его на языке Python с использованием библиотеки NumPy, научиться строить матрицу Якоби и решать систему линейных уравнений на каждой итерации, проверить полученное решение с помощью стандартных библиотечных средств (`scipy.optimize.fsolve`), исследовать влияние начального приближения на сходимость, а также построить линии уровня системы и траекторию итераций.

Теоретические сведения

Постановка задачи

Требуется найти решение системы двух нелинейных уравнений

$$\begin{cases} f_1(x, y) = x^2 + y^2 - 1 = 0, \\ f_2(x, y) = \ln x + y = 0. \end{cases}$$

В векторной форме: $F(X) = 0$, где $X = (x,\, y)^T$. Первое уравнение задаёт окружность радиуса 1 с центром в начале координат; второе — кривую $y = -\ln x$. Из-за логарифма второе уравнение определено только при $x > 0$.

Подставляя $y = -\ln x$ в первое уравнение, получаем

$$x^2 + \ln^2 x = 1, \qquad x > 0.$$

Уравнение имеет два положительных корня: $x = 1$ и $x \approx 0{,}3999$. Следовательно, исходная система имеет два решения:

$$X_1^* = (1{,}0000;\ 0{,}0000), \qquad X_2^* \approx (0{,}3999;\ 0{,}9165).$$

Метод Ньютона для систем нелинейных уравнений

Метод Ньютона для системы $F(X) = 0$ основан на линеаризации вектор-функции в окрестности текущего приближения:

$$F(X) \approx F(X_k) + J(X_k)(X - X_k),$$

где $J(X_k)$ — матрица Якоби системы. Приравнивая линеаризацию нулю, получаем итерационную формулу

$$X_{k+1} = X_k - J(X_k)^{-1} F(X_k).$$

На практике обратную матрицу не вычисляют, а решают систему линейных уравнений

$$J(X_k)\,\Delta_k = -F(X_k), \qquad X_{k+1} = X_k + \Delta_k,$$

что численно устойчивее и реализовано в данной работе через `np.linalg.solve`.

Матрица Якоби

Для рассматриваемой системы

$$J(x, y) = \begin{pmatrix} \dfrac{\partial f_1}{\partial x} & \dfrac{\partial f_1}{\partial y} \\ \dfrac{\partial f_2}{\partial x} & \dfrac{\partial f_2}{\partial y} \end{pmatrix} = \begin{pmatrix} 2x & 2y \\ \dfrac{1}{x} & 1 \end{pmatrix}.$$

Определитель якобиана

$$\det J(x, y) = \frac{2(x^2 - y)}{x}.$$

В корнях $X_1^* = (1, 0)$ и $X_2^* \approx (0{,}3999;\ 0{,}9165)$ определитель отличен от нуля ($\det J \approx 2$ и $\det J \approx -3{,}78$ соответственно), поэтому вблизи обоих корней квадратичная сходимость имеет место.

Критерий остановки

В качестве критерия остановки использовалось условие малости нормы невязки:

$$\|F(X_k)\| < \varepsilon,$$

где $\varepsilon = 10^{-8}$. Дополнительно предусмотрено ограничение на максимальное число итераций (100).

Результаты работы

Задание 1: решение системы методом Ньютона

Для начального приближения $X_0 = (1{,}5;\ 0{,}5)$ метод Ньютона сошёлся за 5 итераций к корню $X_1^* = (1{,}0;\ 0{,}0)$.

Таблица 1: Результат работы метода Ньютона

Параметр	Значение
Начальное приближение X_0	$(1{,}5;\ 0{,}5)$
Найденное значение x^*	1,0000000000
Найденное значение y^*	$-3{,}93 \cdot 10^{-11} \approx 0$
Число итераций	5
Невязка $\|F(X^*)\|$	$5{,}91 \cdot 10^{-11}$

Задание 2: проверка с помощью `scipy.optimize.fsolve`

Таблица 2: Сравнение с `scipy.optimize.fsolve`

Параметр	Значение
Решение `scipy`	$(1{,}0;\ -2{,}81 \cdot 10^{-16})$
Невязка	$4{,}48 \cdot 10^{-16}$
Разница с решением метода Ньютона	$4{,}89 \cdot 10^{-11}$

Задание 3: исследование сходимости

Таблица 3: Зависимость сходимости метода Ньютона от начального приближения

x_0	y_0	Итераций	x^*	y^*	Невязка
0,80	−0,30	4	1,000000	0,000000	$1{,}57 \cdot 10^{-9}$

Из всех рассмотренных начальных приближений метод Ньютона сошёлся к одному и тому же корню $X_1^* = (1,\ 0)$. Второй корень $X_2^* \approx (0{,}3999;\ 0{,}9165)$ не был достигнут, поскольку все стартовые точки лежат в области притяжения корня $(1, 0)$.

Задание 4: визуализация и методика построения графика

Методика построения графика

График строился в среде Python с использованием библиотеки `matplotlib`. Построение выполнялось в следующие шаги:

1. **Построение сетки.** Область определения выбиралась с учётом $x > 0$, поскольку $f_2(x, y) = \ln x + y$ при $x \leq 0$ не определена:

```
x_vals = np.linspace(0.05, 2.5, 400)
y_vals = np.linspace(-2.5, 2.5, 400)
X, Y = np.meshgrid(x_vals, y_vals)
```

2. **Вычисление значений функций** на сетке и построение линий нулевого уровня:

```
F1 = X**2 + Y**2 - 1
F2 = np.log(X) + Y
plt.contour(X, Y, F1, levels=[0], colors='blue', linewidths=2)
plt.contour(X, Y, F2, levels=[0], colors='red',  linewidths=2)
```

   Синяя линия — окружность $x^2 + y^2 = 1$; красная — кривая $y = -\ln x$.

3. **Наложение траектории итераций** по массиву `history`, сохранённому в функции `newton_system`, и выделение корня маркером:

```
history = np.array(history)
plt.plot(history[:, 0], history[:, 1], 'go-',
         linewidth=1.5, markersize=4)
plt.plot(root[0], root[1], 'ks', markersize=10)
```

4. **Формирование легенды.** Поскольку `contour` не поддерживает аргумент `label`, легенда собиралась вручную через объекты `Line2D`:

```
from matplotlib.lines import Line2D
handles = [
    Line2D([0], [0], color='blue',  lw=2, label='x^2 + y^2 - 1 = 0
       '),
    Line2D([0], [0], color='red',   lw=2, label='ln(x) + y = 0'),
    Line2D([0], [0], color='green', marker='o', lw=1.5, label='
                        '),
    Line2D([0], [0], color='black', marker='s', lw=0, markersize
       =10, label='             '),
]
plt.legend(handles=handles, loc='upper right')
```

5. **Оформление осей и сохранение изображения** в файл `newton_system.png` для вставки в отчёт:

```
plt.title('                              (                8)')
plt.xlabel('x'); plt.ylabel('y')
plt.grid(True); plt.axis('equal')
plt.savefig('newton_system.png', dpi=150, bbox_inches='tight')
plt.show()
```

Результат визуализации

На рисунке 1 приведены линии уровня $f_1(x, y) = 0$ и $f_2(x, y) = 0$, а также траектория итераций метода Ньютона из $X_0 = (1{,}5;\ 0{,}5)$.

Из графика видно, что итерации быстро приближаются к точке пересечения кривых — корню $(1, 0)$. На графике также видна вторая точка пересечения $X_2^* \approx (0{,}4;\ 0{,}92)$, в которую траектория из $X_0 = (1{,}5;\ 0{,}5)$ не попадает.

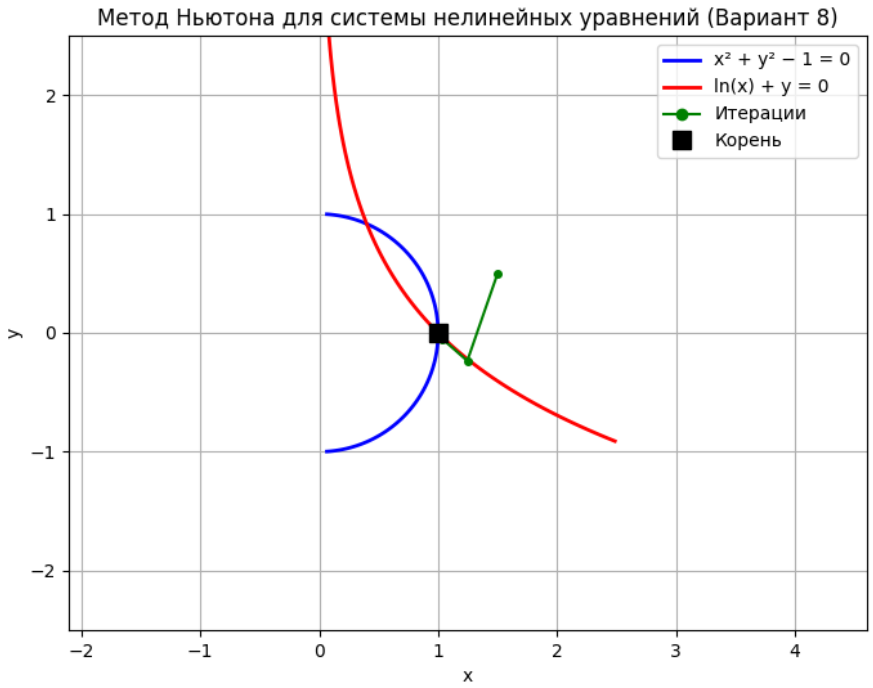


Рис. 1: Линии уровня системы и траектория итераций метода Ньютона

Выводы

- Метод Ньютона для системы реализован и сходится к корню $X_1^* = (1, 0)$ за 5 итераций с невязкой $5{,}91 \cdot 10^{-11}$.
- Проверка через `scipy.optimize.fsolve` подтвердила правильность реализации (разница $4{,}89 \cdot 10^{-11}$).
- При разных начальных приближениях метод устойчиво сходится к корню (1, 0); все исследованные старты лежат в его области притяжения.
- Второй корень $X_2^* \approx (0{,}3999;\ 0{,}9165)$ не был найден из рассмотренных стартов, что подтверждает локальный характер сходимости метода Ньютона.
- Метод Ньютона для систем является эффективным при известной матрице Якоби и начальном приближении из окрестности корня; основной вычислительной нагрузкой является решение системы линейных уравнений с матрицей Якоби на каждой итерации.

Анализ ошибок и рекомендации по улучшению реализации

1. **Область определения логарифма.** Функция $f_2(x, y) = \ln x + y$ определена при $x > 0$. В работе диапазон по x выбран как $[0{,}05;\ 2{,}5]$, чтобы избежать `NaN`.

2. **Параметры `contour`.** Функция `contour` принимает `levels`, но не `lovels`, и не поддерживает аргумент `label`; легенда собирается вручную через `Line2D`.

3. **Создание фигуры.** `plt.figure()` должна вызываться один раз; повторный вызов создаёт новую пустую фигуру.

4. **Совместимость `axis('equal')` и `xlim/ylim`.** При одновременном использовании может возникать предупреждение о игнорировании пределов; при необходимости используют `ax.set_aspect('equal', adjustable='datalim')`.

5. **Критерий остановки.** Использован критерий $\|F(X_k)\| < \varepsilon$. Дополнительно можно контролировать шаг $\|X_{k+1} - X_k\| < \varepsilon$.

6. **Проверка невырожденности якобиана.** В функции `newton_system` предусмотрена проверка $|\det J(X_k)| < 10^{-15}$, при выполнении которой выбрасывается исключение.

Ответы на контрольные вопросы

1. **В чём отличие многомерного метода Ньютона от одномерного?**

   В одномерном случае ищется корень уравнения $f(x) = 0$, и итерация имеет вид

   $$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)},$$

   то есть на каждом шаге выполняется *деление скаляра на скаляр* (производную).

   В многомерном случае ищется решение системы $F(X) = 0$, где $F\colon \mathbb{R}^n \to \mathbb{R}^n$, и итерация записывается как

   $$X_{k+1} = X_k - J(X_k)^{-1} F(X_k),$$

   то есть вместо деления на производную приходится *решать систему линейных уравнений* с матрицей Якоби $J(X_k)$:

   $$J(X_k)\,\Delta_k = -F(X_k), \qquad X_{k+1} = X_k + \Delta_k.$$

   Ключевые отличия:

   - *Скаляр* $f'(x_k)$ заменяется *матрицей* $J(X_k)$.
   - *Деление* f/f' заменяется *решением СЛАУ* $J\Delta = -F$ (обратную матрицу не вычисляют из соображений устойчивости и стоимости).
   - В одномерном случае условие невырожденности — $f'(x^*) \neq 0$; в многомерном — $\det J(X^*) \neq 0$ (невырожденность якобиана).
   - В обоих случаях вблизи простого корня сходимость *квадратичная*: $x_{k+1} - x^* = O\big((x_k - x^*)^2\big)$ и $\|X_{k+1} - X^*\| = O\big(\|X_k - X^*\|^2\big)$.
   - Вычислительная стоимость итерации в многомерном случае существенно выше: $O(n^3)$ операций на решение СЛАУ против $O(1)$ в одномерном.

2. **Что такое матрица Якоби и зачем она нужна?**

   Матрица Якоби (якобиан) системы $F(X) = (f_1, f_2, \ldots, f_n)^T$ в точке X — это матрица первых частных производных:

   $$J(X) = \begin{pmatrix} \dfrac{\partial f_1}{\partial x_1} & \cdots & \dfrac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial f_n}{\partial x_1} & \cdots & \dfrac{\partial f_n}{\partial x_n} \end{pmatrix}.$$

   Для рассматриваемой системы

   $$J(x, y) = \begin{pmatrix} 2x & 2y \\ 1/x & 1 \end{pmatrix}.$$

   Зачем нужна. Якобиан играет роль производной для вектор-функции: он задаёт *линейное приближение* F в окрестности точки X_k:

   $$F(X) \approx F(X_k) + J(X_k)(X - X_k).$$

   На основе этого приближения строится итерационная формула метода Ньютона. Без матрицы Якоби невозможно линеаризовать систему и, следовательно, построить многомерный метод Ньютона. Кроме того, якобиан используется для проверки условий сходимости (невырожденности в корне) и для оценки погрешности.

3. **Как вычисляется следующее приближение в многомерном методе Ньютона?**

   Следующее приближение X_{k+1} вычисляется за две операции:

   (a) **Решение СЛАУ** относительно поправки Δ_k:

   $$J(X_k)\,\Delta_k = -F(X_k).$$

   (b) **Обновление приближения**:

   $$X_{k+1} = X_k + \Delta_k.$$

   В коде это выглядит так:

```
Jx = J(x)
delta = np.linalg.solve(Jx, -Fx)    #                J * delta = -F
x_new = x + delta                   #
```

   Важно, что **обратную матрицу** $J(X_k)^{-1}$ **не вычисляют** — вместо этого решают СЛАУ, что численно устойчивее и дешевле ($O(n^3)$ операций против $O(n^3)$ с большей константой и худшей обусловленностью при явном обращении).

4. **Какие условия гарантируют сходимость метода Ньютона?**

   Достаточные условия *локальной* сходимости метода Ньютона:

   (a) **Существование корня.** В некоторой окрестности точки X_0 существует корень X^*, т.е. $F(X^*) = 0$.

   (b) **Непрерывность якобиана.** Матрица Якоби $J(X)$ непрерывна в окрестности корня.

   (c) **Невырожденность якобиана в корне.** $\det J(X^*) \neq 0$, т.е. корень *простой*. В нашем случае $\det J(X_1^*) = 2 \neq 0$.

   (d) **Близость начального приближения.** X_0 достаточно близко к X^* (принадлежит области притяжения корня).

   При выполнении этих условий последовательность $\{X_k\}$ сходится к X^*, причём *квадратично*:

   $$\|X_{k+1} - X^*\| \leq C\,\|X_k - X^*\|^2.$$

5. **Почему метод Ньютона может не сойтись при неудачном начальном приближении?**

   Метод Ньютона является *локально сходящимся*: гарантия сходимости имеет место только в достаточно малой окрестности корня. При неудачном начальном приближении возможны следующие проблемы:

   - **Расходимость.** Последовательность $\{X_k\}$ уходит на бесконечность, так как линеаризация в текущей точке уводит приближение далеко от корня.
   - **Циклические траектории.** Итерации могут «зацикливаться», не приближаясь к корню.

- **Сходимость к другому корню.** Если система имеет несколько корней, метод может сойтись не к тому, который нужен (в нашей системе это как раз наблюдается: из всех рассмотренных стартов сходятся к корню $(1, 0)$, а не к $(0{,}4;\ 0{,}92)$).
- **Вырожденность якобиана.** Если $\det J(X_k) \approx 0$, решение СЛАУ становится неустойчивым; при $\det J = 0$ метод вообще не определён.
- **Область определения.** Начальное приближение может вывести итерацию за пределы области определения функции (например, для $\ln x$ — в область $x \leq 0$), что приводит к `NaN` и остановке.

Для функции из нашей системы производная якобиана не обращается в нуль в корнях, поэтому при удачном выборе X_0 (например, $X_0 = (1{,}5;\ 0{,}5)$) метод сходится устойчиво.

6. **Как проверить, что матрица Якоби не вырождена?**

   Матрица Якоби $J(X_k)$ невырождена, если её определитель отличен от нуля. Существуют три уровня проверки:

   (a) **Проверка определителя.** Вычислить $\det J(X_k)$ и убедиться, что $|\det J(X_k)| > \delta$ для некоторого малого порога δ (например, $\delta = 10^{-15}$). В коде:

```
if abs(np.linalg.det(Jx)) < 1e-15:
    raise ValueError("
                           ")
```

   (b) **Проверка числа обусловленности.** Число обусловленности

$$\mathrm{cond}(J) = \|J\| \cdot \|J^{-1}\|$$

   характеризует близость матрицы к вырожденной. Большое значение $\mathrm{cond}(J) \gg 1$ сигнализирует о плохой обусловленности: даже при $\det J \neq 0$ метод Ньютона может работать неустойчиво. В коде:

```
if np.linalg.cond(Jx) > 1e12:
    raise ValueError("
                                ")
```

   (c) **Аналитическая проверка в корне.** Найти корень X^* и вычислить $\det J(X^*)$. Для нашей системы

$$\det J(x, y) = \frac{2(x^2 - y)}{x}.$$

   В корне $X_1^* = (1, 0)$:

$$\det J(1, 0) = \frac{2(1 - 0)}{1} = 2 \neq 0.$$

   В корне $X_2^* \approx (0{,}3999;\ 0{,}9165)$:

$$\det J(X_2^*) \approx \frac{2(0{,}16 - 0{,}9165)}{0{,}3999} \approx -3{,}78 \neq 0.$$

   Оба корня простые, поэтому в окрестности каждого из них квадратичная сходимость имеет место.